ATOSFleetManagement: Architecture and Development Guide

Purpose

```
ATOSFleetManagement is a lightweight ATOS runtime focused on truck monitoring and
  command flow. It is built around TruckObjectControl and TruckObjectGUI and
  intentionally does not start the full ATOS scenario stack.
```

Runtime scope

```
Included core runtime pieces:
- truck_object_control
- truck_object_gui (FleetManagement mode)
- foxglove_bridge or rosbridge_websocket
- optional atos_truck_simulator instances
Not started in this mode:
- OpenScenarioGateway
- JournalControl
- EsminiAdapter
- ObjectControl and the full scenario-execution path
```

Launch entrypoint

```
Main launch file:
ros2 launch atos launch_atosfleetmanagement.py
Important launch arguments:
- insecure:=True|False
- foxbridge:=True|False
- with_truck_simulator:=True|False
- cot_tls_require_client_cert:=True|False
- cot_tls_cert_path:=<path>
- cot_tls_key_path:=<path>
- cot_tls_ca_path:=<path>
```

Component overview

```
TruckObjectControl receives truck state/COT input, tracks route progress, and
  publishes speed commands.
TruckObjectGUI runs NiceGUI in FleetManagement mode and renders a route-based map
  view with live truck overlays from /atos/truck_objects/state.
Foxglove Bridge or Rosbridge provides websocket access depending on the selected
  launch mode.
ATOS truck simulators can be enabled for local development without real truck input.
```

Default ports and endpoints

```
- TruckObjectGUI: http://localhost:8420
- Foxglove Bridge TLS/non-TLS port: 8765
- Rosbridge TLS/non-TLS port: 9090
- TruckObjectControl COT TCP listener: tcp://0.0.0.0:8114
```

Topics and messages

```
- Input topic placeholder: /atos/truck_objects/cot
- Speed command topic: /atos/truck_objects/speed_command
- Live truck state topic: /atos/truck_objects/state
Current placeholder COT payload used for development:
id=<truck_id>;distance_m=<value>;tcp_connected=<0|1>
```

Route and map data

```
Default route asset in the repository:
conf/conf/RuralRoad_center_of_driving_lane_ccw.geojson
```

```
TruckObjectControl and the optional simulators use this route as their default
  trajectory_geojson_path.
The GUI loads the GeoJSON route and overlays live truck states in the
  FleetManagement map view.
```

Build workflow

```
Supported native installations:
- Ubuntu 20.04 with ROS 2 Foxy
- Ubuntu 22.04 with ROS 2 Humble
- Ubuntu 24.04 with ROS 2 Jazzy
Always source ROS before building:
source /opt/ros/<foxy|humble|jazzy>/setup.bash
Manual workspace build:
cd <ATOS_WORKSPACE>
colcon build --symlink-install --cmake-args -DWITH_TRUCK_OBJECT_CONTROL=ON
Helper build script:
cd <ATOS_REPO_ROOT>
./scripts/build_atosfleetmanagement.sh
The helper script builds atos and atos_gui and disables legacy modules not used in
  this runtime.
```

Local launch workflow

```
source /opt/ros/<foxy|humble|jazzy>/setup.bash
source <ATOS_WORKSPACE>/install/setup.bash
ros2 launch atos launch_atosfleetmanagement.py foxbridge:=False insecure:=True
Use with_truck_simulator:=True to start the built-in simulator nodes.
```

Docker runtime

```
Dedicated compose file:
docker-compose-fleetmanagement.yml
Start with docker compose:
docker compose -f docker-compose-fleetmanagement.yml up --build
Optional environment flags:
- WITH_TRUCK_SIMULATOR=True|False
- INSECURE=True|False
- FOXBRIDGE=True|False
- COT_TLS_REQUIRE_CLIENT_CERT=True|False
- COT_TLS_CERT_PATH=<path>
- COT_TLS_KEY_PATH=<path>
- COT_TLS_CA_PATH=<path>
Runtime container entrypoint:
./scripts/run_atosfleetmanagement.sh
```

Service deployment

```
Service assets:
- scripts/atosfleetmanagement.env.example
- scripts/atosfleetmanagement.service
Typical deployment flow:
1. Copy the repository to the target host, for example /opt/atos
2. Install /etc/default/atosfleetmanagement from the example env file
3. Install the systemd unit from scripts/atosfleetmanagement.service
4. Enable and start the service with systemctl
```

Developer utilities

```
- scripts/build_atosfleetmanagement.sh: focused build helper
- scripts/run_atosfleetmanagement.sh: runtime launcher used by Docker/service
```

  deployment
- scripts/sync_to_buildserver.sh: sync helper for deployment/build server workflows

Design notes

This runtime is intentionally smaller than the full ATOS stack. The goal is a
  practical truck-oriented control and visualization environment with fewer moving
  parts for development, deployment, and operations.